Day 6 – Assignment – LINQ with C#

Complete this exercise for learning LINQ. It is Insurance Policy Management System – LINQ related C# exercise of duration 4-5 Hours approximately.

Prerequisites: C#, Classes and Objects, Collections (List<T>), Lambda Expressions, Anonymous Types

Note: No Database, Use List<T> with sample data, Entire exercise should be in a Console Application.

Business Scenario: ABC Insurance maintains information about: Customers, Insurance Policies, Claims, Agents, Premium Payments. The management wants different reports using LINQ instead of loops. Your task is to generate these reports.

Model Classes

```
public class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public string City { get; set; }
    public int Age { get; set; }

}

public class Policy
{
    public int PolicyId { get; set; }
    public int CustomerId { get; set; }
    public string PolicyType { get; set; }
    public decimal Premium { get; set; }
    public decimal CoverageAmount { get; set; }
    public DateTime StartDate { get; set; }
    public bool IsActive { get; set; }

}

public class Claim
{
    public int ClaimId { get; set; }
    public int PolicyId { get; set; }
    public decimal ClaimAmount { get; set; }
    public string Status { get; set; }
}
public class Agent
{
    public int AgentId { get; set; }
    public string AgentName { get; set; }
    public string Region { get; set; }
}

public class Payment
{
    public int PaymentId { get; set; }
    public int PolicyId { get; set; }
    public decimal Amount { get; set; }
    public DateTime PaymentDate { get; set; }
}
```

Sample Data : Customers

```
List<Customer> customers = new()
{
    new Customer{CustomerId=1,Name="Amit Sharma",City="Delhi",Age=32},
    new Customer{CustomerId=2,Name="Priya Singh",City="Mumbai",Age=29},
    new Customer{CustomerId=3,Name="Rahul Verma",City="Hyderabad",Age=45},
    new Customer{CustomerId=4,Name="Sneha Kapoor",City="Delhi",Age=38},
    new Customer{CustomerId=5,Name="Vikas Mehta",City="Pune",Age=50},
    new Customer{CustomerId=6,Name="Neha Gupta",City="Hyderabad",Age=27},
    new Customer{CustomerId=7,Name="Ankit Jain",City="Bangalore",Age=41},
    new Customer{CustomerId=8,Name="Pooja Nair",City="Chennai",Age=35},
    new Customer{CustomerId=9,Name="Rohit Das",City="Kolkata",Age=48},
    new Customer{CustomerId=10,Name="Meera Joshi",City="Mumbai",Age=31}
};
```

Sample Data : Policies

```
List<Policy> policies = new()
{
    new
Policy{PolicyId=101,CustomerId=1,PolicyType="Health",Premium=12000,CoverageAmount
=500000,StartDate=new DateTime(2025,1,1),IsActive=true},
    new
Policy{PolicyId=102,CustomerId=2,PolicyType="Life",Premium=18000,CoverageAmount=1
000000,IsActive=true},
    new
Policy{PolicyId=103,CustomerId=3,PolicyType="Motor",Premium=9000,CoverageAmount=3
00000,IsActive=false},
    new
Policy{PolicyId=104,CustomerId=4,PolicyType="Health",Premium=15000,CoverageAmount
=700000,IsActive=true},
    new
Policy{PolicyId=105,CustomerId=5,PolicyType="Life",Premium=25000,CoverageAmount=1
500000,IsActive=true},
    new
Policy{PolicyId=106,CustomerId=6,PolicyType="Travel",Premium=4000,CoverageAmount=
100000,IsActive=true},
    new
Policy{PolicyId=107,CustomerId=7,PolicyType="Motor",Premium=8500,CoverageAmount=3
50000,IsActive=true},
    new
Policy{PolicyId=108,CustomerId=8,PolicyType="Health",Premium=13000,CoverageAmount
=600000,IsActive=false},
    new
Policy{PolicyId=109,CustomerId=9,PolicyType="Life",Premium=21000,CoverageAmount=1
200000,IsActive=true},
    new
Policy{PolicyId=110,CustomerId=10,PolicyType="Motor",Premium=9500,CoverageAmount=
400000,IsActive=true}
};
```

Sample Data: Claims

```
List<Claim> claims = new()
{
    new Claim{ClaimId=1,PolicyId=101,ClaimAmount=50000,Status="Approved"},
    new Claim{ClaimId=2,PolicyId=103,ClaimAmount=120000,Status="Pending"},
    new Claim{ClaimId=3,PolicyId=104,ClaimAmount=45000,Status="Rejected"},
    new Claim{ClaimId=4,PolicyId=105,ClaimAmount=300000,Status="Approved"},
    new Claim{ClaimId=5,PolicyId=107,ClaimAmount=70000,Status="Approved"},
    new Claim{ClaimId=6,PolicyId=110,ClaimAmount=90000,Status="Pending"}
};
```

Sample Data: Payments

```
List<Payment> payments = new()
{
    new
Payment{PaymentId=1,PolicyId=101,Amount=12000,PaymentDate=DateTime.Today.AddMonth
s(-5)},
    new
Payment{PaymentId=2,PolicyId=102,Amount=18000,PaymentDate=DateTime.Today.AddMonth
s(-4)},
    new
Payment{PaymentId=3,PolicyId=104,Amount=15000,PaymentDate=DateTime.Today.AddMonth
s(-3)},
    new
Payment{PaymentId=4,PolicyId=105,Amount=25000,PaymentDate=DateTime.Today.AddMonth
s(-2)},
    new
Payment{PaymentId=5,PolicyId=107,Amount=8500,PaymentDate=DateTime.Today.AddMonths
(-1)},
    new Payment{PaymentId=6,PolicyId=110,Amount=9500,PaymentDate=DateTime.Today}
};
```

LINQ Exercises

Level 1 (Filtering)

1. Display all active policies.
2. Find customers from Delhi.
3. Find Health policies.
4. Display policies whose premium is greater than ₹15,000.
5. Display customers older than 40.

Level 2 (Sorting)

6. Sort customers by Name.
7. Sort policies by Premium descending.
8. Sort policies by Coverage Amount.
9. Display newest policies first.
10. Sort customers by City then Name.

Level 3 (Projection)

11. Display only Customer Name and City.
12. Display PolicyId and Premium.
13. Display Customer Name with Age Category.
14. Create anonymous objects containing Policy Type and Premium.
15. Display customer names in uppercase.

Level 4 (Aggregation)

16. Total Premium Collection.
17. Average Premium.
18. Maximum Coverage Amount.
19. Minimum Premium.
20. Count Active Policies.

Level 5 (Grouping)

21. Group policies by Policy Type.
22. Count policies in each type.
23. Total premium collected by policy type.
24. Group customers by City.
25. Average premium by policy type.

Level 6 (Joins)

26. Display Customer Name with Policy Type.
27. Display Customer Name with Premium.
28. Display Customer Name and Coverage Amount.
29. Display Customer Name and Policy Status.
30. Display Customer Name, Policy Type, Premium.

Level 7 (Advanced)

31. Customers without any policy.
32. Policies without claims.
33. Policies having approved claims.
34. Customers with multiple policies.
35. Highest premium policy.

Level 8 (Quantifiers)

36. Is there any inactive policy?
37. Are all policies active?
38. Does any customer belong to Chennai?
39. Does every policy have premium > ₹3000?
40. Are there any rejected claims?

Level 9 (Element Operators)

41. First active policy.
42. Last Health policy.
43. Single customer with CustomerId=5.
44. FirstOrDefault for Travel policy.
45. ElementAt(3).

Level 10 (Set Operators)

46. Distinct cities.
47. Union of two city lists.
48. Intersect two policy type lists.
49. Except operation.
50. Distinct policy types.

Level 11 (Partitioning)

51. Take first five customers.
52. Skip first three customers.
53. TakeWhile premium < ₹20,000.

54. SkipWhile age < 35.
55. Pagination (Take + Skip).

Level 12 (Conversion)

56. Convert to Dictionary.
57. Convert to Lookup.
58. Convert to Array.
59. Convert to List.
60. Convert anonymous objects into a list.

Extra Challenges

61. Top 3 customers paying highest premium.
62. Customer having maximum coverage.
63. Total approved claim amount.
64. City-wise premium collection.
65. Policy-wise claim summary.
66. Customer with highest number of policies.
67. Average customer age by city.
68. Premium collected month-wise.
69. Customers having both Life and Health policies.

70. Dashboard showing:

- Total Customers
- Total Policies
- Active Policies
- Total Premium
- Total Claims
- Approved Claims
- Total Coverage
- Average Premium

ꕥ Debug your doubts, deploy your success ꕥ